

Per-agent settings locations — user vs. admin-managed

For each agent ai-protect currently wires (via `ConfigDelta / ManagedEnvVar` — see `broker-status-file-schema.md`), this doc lists the **user-settings path ai-protect writes today**, and whether a real **admin-managed** location exists that could carry the same registration instead — a meaningfully different trust/deployment model (MDM/GPO-controlled, not the logged-in user's own account) worth moving to where it's genuinely available.

Headline finding, read this before the table: a managed-settings file existing is not the same question as whether it can carry *what we write*. Several tools have a real, root-owned managed tier that can only **allow/deny an MCP server by name or URL** — it cannot **register** one, which is what our `ConfigDelta` does. Moving to "managed settings" only helps where the managed schema actually accepts a server registration (or an `env` /proxy key for the LLM leg), not wherever a managed file exists at all. The three tiers below are sorted on exactly that distinction.

Sourced from each vendor's own current docs (cited per row); anything not independently re-verified against a live install is marked accordingly — this is the same discipline the rest of this integration follows: verify against current docs, don't carry forward a stale assumption.

Tier 1 — the managed channel can carry a real MCP/env registration today

Agent	User path (today)	Managed path	What it can carry
Claude Code	<code>~/.claude.json</code> , <code>~/.claude/settings.json</code>	<code>managed-mcp.json</code> at <code>/Library/Application Support/ClaudeCode/</code> (macOS), <code>/etc/claude-code/</code> (Linux/WSL), <code>C:\Program Files\ClaudeCode\</code> (Windows — not the legacy <code>C:\ProgramData\...</code> path some older references cite)	Full MCP registration — same <code>{"mcpServers": {...}}</code> schema as the user file. Deploying it gives the admin <i>exclusive</i> control (user/project/plugin MCP loads and <code>claude mcp add</code> are disabled). <code>env</code> is also a real managed-settings.json key — the natural channel for our LLM-leg's <code>ANTHROPIC_BASE_URL</code> /C A-path vars too. docs

Agent	User path (today)	Managed path	What it can carry
Gemini CLI	<code>~/.gemini/settings.json</code>	<code>settings.json</code> under <code>/etc/gemini-cli/</code> , <code>C:\ProgramData\gemini-cli\</code> , <code>/Library/Application Support/GeminiCli/</code> (overridable via <code>GEMINI_CLI_SYSTEM_SETTINGS_PATH</code>)	mcpServers is directly settable at this tier and sits <i>above</i> user and workspace settings — merges by name, system definition wins. Our adapter's proxy key (the ForwardProxy leg) is unconfirmed at this tier specifically — same settings.json shape, so plausible, not yet verified. docs
Amp	(per-editor MCP config)	<code>managed-settings.json</code> at <code>/Library/Application Support/ampcode/</code> , <code>/etc/ampcode/</code> , <code>%ProgramData%\ampcode\</code>	Can set amp.mcpServers directly — a real registration channel, not just allow/deny — plus <code>amp.mcpPermissions</code> . announcement
OpenAI Codex CLI	<code>~/.codex/config.toml</code>	<code>requirements.toml</code> (hard constraints) + <code>managed_config.toml</code> (managed defaults) at <code>/etc/codex/</code> (macOS/Linux), <code>%ProgramData%\OpenAI\Codex\</code> (Windows)	<code>[mcp_servers]</code> identity rules are enforceable via <code>requirements.toml</code> — described as constraint/enforcement rather than confirmed full free-form registration; verify the exact semantics against a live install before relying on it to <i>add</i> a server rather than <i>constrain</i> one. docs
OpenCode	<code>~/.config/opencode/opencode.json</code>	<code>/Library/Application Support/opencode/</code> , <code>/etc/opencode/</code> , <code>%ProgramData%\opencode\</code> , plus a macOS MDM domain (<code>ai.opencode.managed</code>)	Highest priority, documented as non-overridable. Exact key-level MCP support not itemized in what was checked — verify before relying on it for registration specifically. docs

Tier 2 — a managed channel exists, but it can only allow/deny, not register

Agent	User path (today)	Managed path	The gap
GitHub Copilot	<code>~/.copilot</code>	<code>managed-settings.json</code> at <code>/Library/Application Support/GitHubCopilot/</code> , <code>%ProgramFiles%\GitHubCopilot\</code> , <code>/etc/github-copilot/</code> (also MDM/server-managed)	Only <code>allowedMcpServers / deniedMcpServers</code> (by <code>serverName / serverUrl / serverCommand</code>) — no key to register a new server. docs
VS Code (Copilot's editor host)	n/a — routed to the <code>copilot</code> adapter, VS Code itself is dormant	ADMX/registry/ <code>.mobileconfig / etc/vscode/policy.json</code> , fixed allowlist of policies VS Code's own source registers	<code>ChatAllowedMcpServers / ChatDeniedMcpServers</code> allow/deny only; no policy lets an admin force an arbitrary <code>settings.json</code> key (open feature request, vscode#312764); no <code>http.proxy</code> policy at all.
Cursor	<code>~/.cursor/mcp.json</code> (MCP), <code>~/.cursor/cli-config.json</code> -adjacent (LLM leg)	ADMX/ <code>.mobileconfig / ~ / .cursor/policy.json</code> — only 6 policies, none MCP/model-related	MCP/model control is <code>~/.cursor/permissions.json</code> (<code>mcpAllowlist</code>) — MDM-distributable, but it is still a user-home file , the same trust tier as <code>mcp.json</code> itself, not a root-owned location. The real org-level control is the cloud Admin Console (server-side, no local artifact to target). docs
Windsurf / Devin Desktop	<code>~/.codeium/windsurf/mcp_config.json</code> (MCP), its own <code>settings.json</code> (LLM leg, <code>http.proxy</code>)	Real ADMX/ <code>.mobileconfig / etc/windsurf/policies/policy.json</code> , rebranded from Windsurf's own, computer-level beats user-level	Documented policies are <code>AllowedExtensions</code> , <code>EnableTelemetry</code> , <code>EnableFeedback</code> , <code>UpdateMode</code> only — no MCP policy at all . docs
Kiro	<code>~/.kiro/settings/mcp.json</code> (MCP), its own <code>settings.json</code> (LLM leg, <code>http.proxy</code>)	Registry/ <code>.mobileconfig / etc/kiro/policy.json</code> exists but the only documented value is <code>ExtensionGalleryServiceUrl</code> — no managed <code>settings.json</code> equivalent.	Enterprise MCP/model control is account-level via the Kiro console + IAM Identity Center, and Kiro's own docs concede it's client-enforced and circumventable by a local admin. docs

Agent	User path (today)	Managed path	The gap
Antigravity	<code>~/.gemini/config/mcp_config.json</code>	None found — enterprise docs cover only Gemini Enterprise licensing/VPC-SC/logging/BYOID, no GPO/registry/plist/ADMX at all	Treat as genuinely absent, not merely undocumented — nothing in Google's own enterprise page for this product resembles a managed-settings file.

Tier 3 — no managed tier exists; user settings are the only option

Agent	Path	Note
Devin CLI	<code>~/.config/devin/config.json</code>	An org/enterprise tier is documented at the top of config precedence, but administered entirely in Devin's cloud console — no local system path exists to target.
Zed	<code>~/.config/zed/settings.json</code>	Zed Business's admin controls are server-side org toggles (collaboration, model provider, data controls) enforced through Zed's own infrastructure — not a local settings file.
Aider	<code>~/.aider.conf.yml</code>	No managed tier — user file, repo root, cwd, or <code>AIDER_*</code> env vars only.
OpenClaw	<code>~/.openclaw/openclaw.json</code>	No managed tier found; its env vars (<code>OPENCLAW_CONFIG_PATH</code> , <code>OPENCLAW_STATE_DIR</code>) are for multi-instance isolation, not policy.
Grok (official Grok Build)	<code>~/.grok/user-settings.json</code> (the path our <code>grok.rs</code> adapter targets)	A Codex-shaped managed tier exists (<code>/etc/grok/managed_config.toml + requirements.toml</code>) but MCP support there is not documented — don't assume parity with Codex's <code>[mcp_servers]</code> handling without checking. Also worth re-flagging: this session found the <i>npm package</i> commonly associated with "Grok CLI" (<code>@vibe-kit/grok-cli</code>) is an unrelated third-party tool, not xAI's own — the managed-tier docs cited here are for the official product, <code>grok</code> /Grok Build.
Continue	<i>(now discontinued — acqui-hired by Anysphere/Cursor June 2026, repo read-only; see the Fork Ledger)</i>	Not researched — moot given the product's status.
Amazon Q Developer CLI	<i>(EOL)</i>	Out of scope — superseded by Kiro, already covered above.

Recommendation

1. **Claude Code, Gemini CLI, Amp** — move both legs (MCP registration and, where the leg exists, the LLM redirect / env vars) to the managed channel now. This is a straightforward win: real admin-owned files, confirmed to accept the actual registration our `ConfigDelta` carries today, not just an allow/deny gate.
2. **Codex, OpenCode** — same direction, but verify the exact managed-key semantics against a live install first (both describe their managed tier in terms suggesting *constraint/defaults* more than *free registration* — worth confirming before assuming parity with Claude Code's `managed-mcp.json`).
3. **Copilot, VS Code, Cursor, Windsurf/Devin Desktop, Kiro** — do **not** move MCP registration to the managed channel; it structurally can't carry it (allow/deny only). The user-settings path stays the only real registration surface for these five, regardless of deployment preference. If reducing user-writable surface area matters here, the lever is different: `ChatAllowedMcpServers` -style allowlisting narrows what a *user* could add on their own, but doesn't move where we write.
4. **Antigravity, Devin CLI, Zed, Aider, OpenClaw, Grok** — no change possible; user settings remain the only option for all of these.

None of the managed tiers surveyed — Tier 1 included — were confirmed against a **live enrolled install** during this pass; this is a docs-only survey. Before shipping a managed-settings delivery path, validate write access (these are root/admin-owned locations — ai-protect already runs privileged, so this is a path change, not a new privilege requirement) and confirm precedence behavior matches what's documented (e.g. Claude Code's `managed-mcp.json` disabling *every other* MCP source once present, which changes what "adding our server" means in practice — it becomes "the only server," not "one more server").